DAIFEND

SECURING THE AI RUNTIME

PLATFORM GUIDE

FEATURES & HOW TO USE • 2026

Features & how to use

Daifend is an AI-native security platform: a **Next.js console** plus **production-structured backends** (API gateway, auth, telemetry, memory integrity, agent runtime, threat/RAG, self-healing, notifications), with optional **ClickHouse**, **Kafka/Redpanda**, **NATS**, **Qdrant**, **OPA**, **OpenTelemetry**, and **gRPC** for memory analyze.

CONSOLE (UI)

- **Dashboard** — trust timeline, drift matrix, threat ticker, runtime posture, self-healing feed.
- **Modules** — AI Memory Security, Agent Runtime, Threat Intel, Incidents, Self-Healing, Research Lab, Live Simulation, Architecture, API Integrations, Settings.
- **Telemetry** — `demo` (Node mock) or `live` (Python service + Socket.IO).
- **Simulation** — investor-style attack spike + healing visualization.

BACKEND & DATA PLANE

- **API gateway** — JWT / OIDC JWKS, Redis rate limits, optional mTLS to engines, optional gRPC to memory.
- **Engines** — Memory integrity (numpy + optional Qdrant), agent policy + **OPA**, RAG scan, threat correlate, healing workflows.
- **Persistence** — PostgreSQL + Alembic; telemetry to ClickHouse; streams via NATS/Kafka.
- **Web DB access** — Prisma in `apps/web`; `GET /api/health/db` when `DATABASE_URL` is set.

How to use — quick start

PREREQUISITES

Node 20+, Python 3.12+ (for engines), Docker (optional, full stack).

1) UI + DEMO TELEMETRY (LIGHTWEIGHT)

```
cp .env.example .env
npm install
npm run dev
npm run telemetry:demo --workspace=@daifend/web
```

Open `http://127.0.0.1:3000`. Set `NEXT_PUBLIC_TELEMETRY_URL` if the mock server is not on port 4001.

2) FULL STACK (DOCKER COMPOSE)

```
docker compose up -d --build
```

URL / port	Purpose
`127.0.0.1:3000`	Web console
`127.0.0.1:8080`	API gateway (`/health`)
`127.0.0.1:4001`	Telemetry (Socket.IO)
`127.0.0.1:16686`	Jaeger (traces)
`127.0.0.1:8181`	OPA (agent policy)
`127.0.0.1:8123`	ClickHouse HTTP
`127.0.0.1:19092`	Kafka API (Redpanda)

Compose sets `NEXT_PUBLIC_DAIFEND_MODE=live`, engine service tokens, and `MEMORY_GRPC_TARGET` for analyze via gRPC. See `docs/LOCAL_DEV.md` and `docs/PHASE3.md`.

3) DATABASE MIGRATIONS (POSTGRES)

```
cd packages/daifend-core && pip install -e .
export DATABASE_URL=postgresql://daifend:daifend@localhost:5432/daifend
alembic upgrade head
```

4) PRISMA CLIENT (WEB)

```
export DATABASE_URL=postgresql://daifend:daifend@localhost:5432/daifend
npm run db:generate --workspace=@daifend/web
```

Health: `GET /api/health/db` on the Next app.

API gateway (summary)

Base URL: `http://127.0.0.1:8080` (adjust for deploy). Send `X-Tenant-Id`. Authenticate with `Authorization: Bearer <token>` from `POST /v1/oauth/token`, or (dev) `X-Internal-Token` matching `INTERNAL_SERVICE_TOKEN`.

POST	`/v1/oauth/token` — obtain access token
POST	`/v1/memory-integrity/analyze` — memory / embedding integrity (gRPC if `MEMORY_GRPC_TARGET` set)
POST	`/v1/agent-runtime/validate-action` — tool / policy (+ OPA when configured)
POST	`/v1/rag/scan-document`, `/v1/threats/correlate`
POST	`/v1/self-healing/workflows`, `/v1/notify`

TypeScript contracts: package `@daifend/types`; client helper: `@daifend/sdk`. Full detail: `docs/API.md`.

Documentation index

- `README.md` — repo layout & commands
- `docs/ARCHITECTURE.md` — system diagram & data flow
- `docs/LOCAL_DEV.md` — ports, env, debugging
- `docs/API.md` — gateway routes & auth
- `docs/HARDENING.md` — OTel, Redis, OIDC, ClickHouse, Kafka
- `docs/PHASE3.md` — service tokens, mTLS hooks, gRPC, OPA, Prisma
- `docs/GRPC.md` — proto & codegen